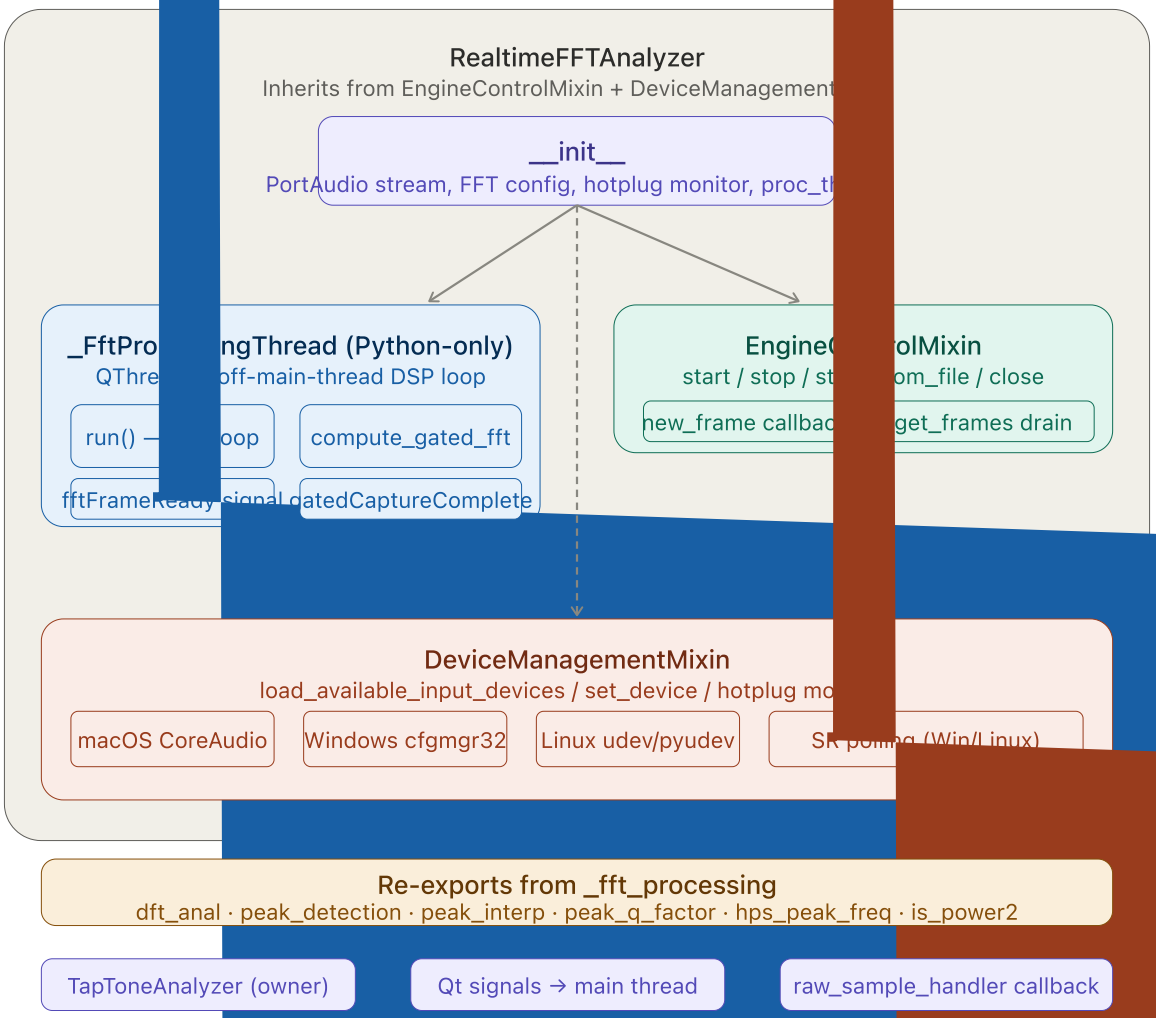
GuitarTap Python — RealtimeFFTAnalyzer Architecture

realtime_fft_analyzer · _engine_control mixin · _device_management mixin · _fft_processing module

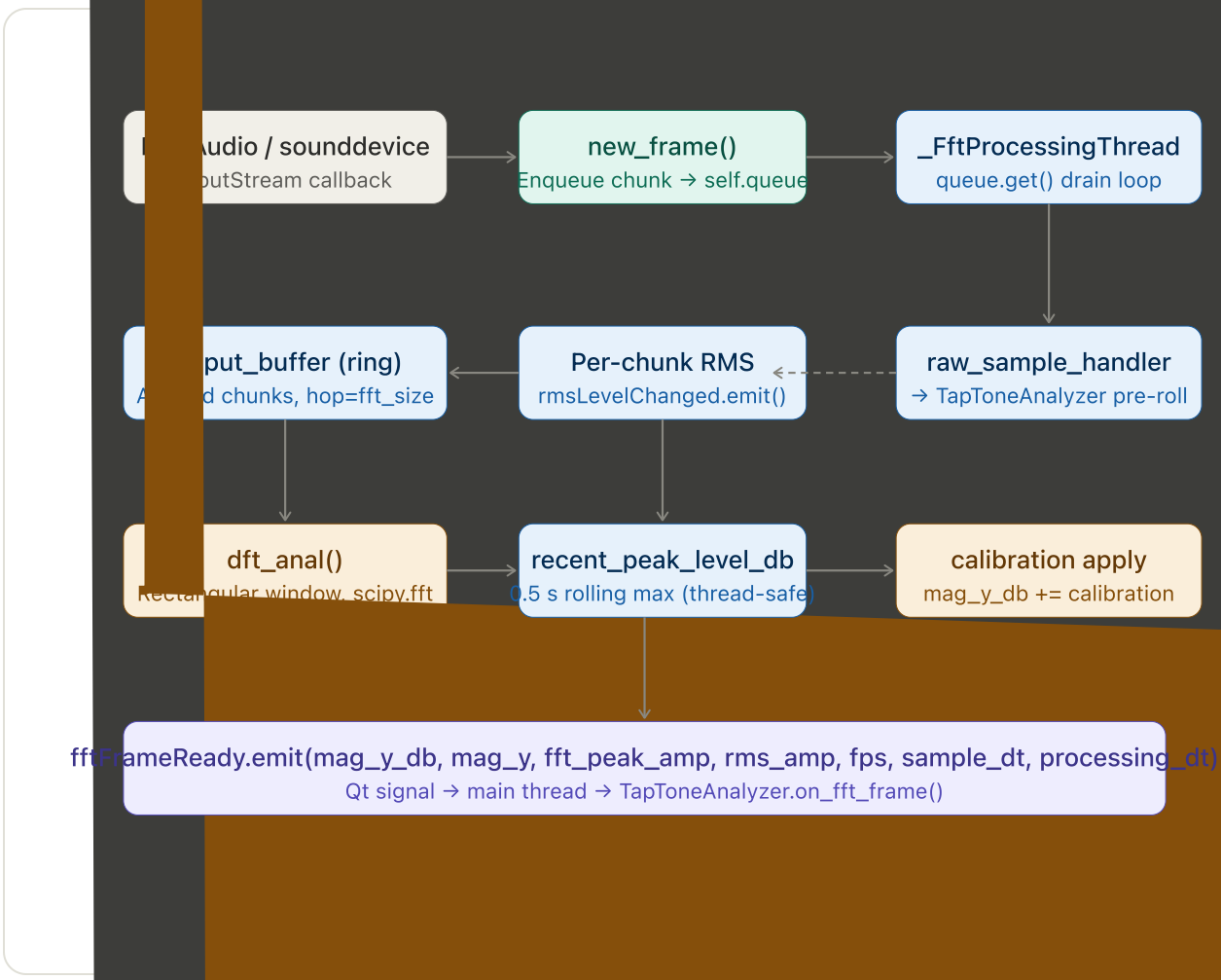
1. Full class architecture

`RealtimeFFTAnalyzer` inherits from two mixins. The base file holds a few shared properties and `__init__`, plus the Python-only `_FftProcessingThread` (a `QThread` that handles off-main-thread DSP — there is no Swift equivalent). The module also re-exports FFT functions from `_fft_processing` for backward compatibility.



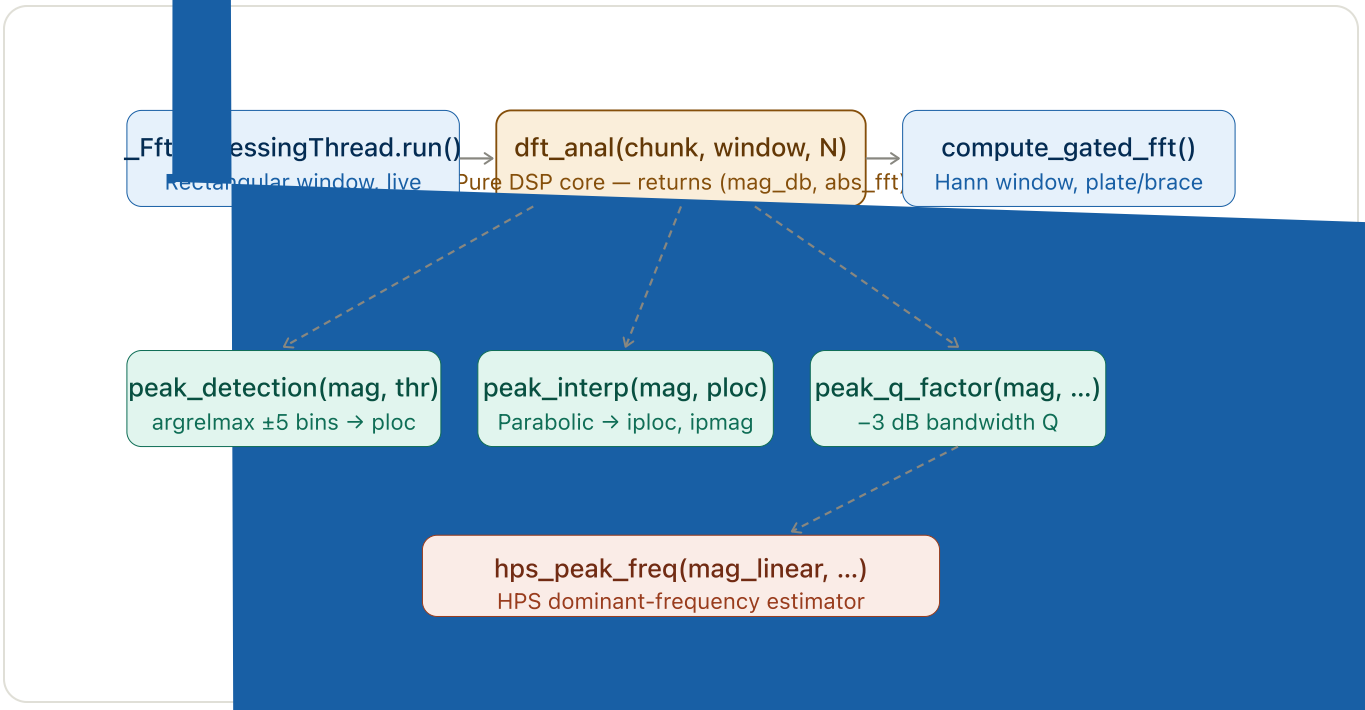
2. Audio processing pipeline

P[illegible]udio [illegible] `new_frame` on every block, which enqueues the chunk into `self.queue`. [illegible]roce[illegible]ngThread.run()` drains the queue, maintains the ring buffer, computes per-chunk RMS, fires [illegible]_sample_handler` on every chunk (so `TapToneAnalyzer` gets the pre-roll and gated accum[illegible] `fftFrameReady` [illegible] The Swift equivalent uses AVAud[illegible] `audioProcessingQueue`



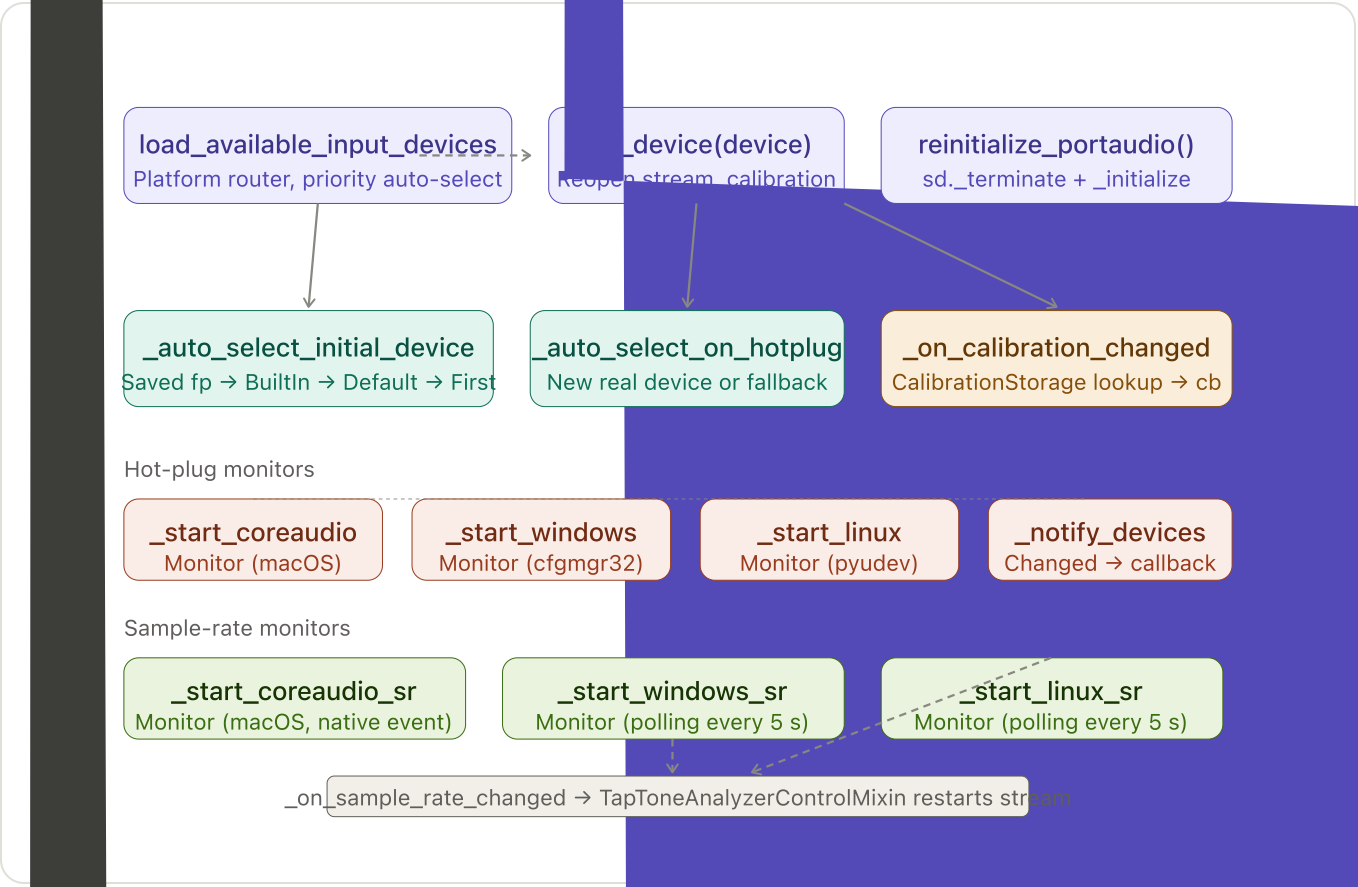
3. FFT processing functions (_fft_processing module)

The mo[illegible]vides pure side-effect-free functions. `dft_anal` is the direct counterpart to Swift's `computeFFT(on:)` — both are the pure DSP core used by the test suite. Swift uses vDSP via deinter[illegible]plit-complex format; Python uses `scipy.fft` with a zero-phase rotation trick. Both paths a[illegible]alent: rectangular window for the live display path, Hann window for the gated plate/b[illegible]n.



4. Device management and hot-plug monitoring

`load_available_input_devices` queries PortAudio and applies the same four-priority auto-selection policy as Swift's `loadAvailableInputDevicesMacOS`. `set_device` closes and reopens the stream, auto-loads calibration via the `_on_calibration_changed` callback, and registers the per-device sample-rate listener. Hot-plug monitoring is platform-split: CoreAudio listener on macOS, `CM_Register_Notification` on Windows, `pyudev` on Linux. Sample-rate change detection uses a CoreAudio listener on macOS and a polling thread on Windows/Linux (no native event).



Ke chitecture notes

Sw s Python structural differences

C rn	Swift	Python
A e	AVAudioEngine (AVFoundation)	sounddevice.InputStream (PortAudio)
C in D	AVAudioEngine audio processing queue (implicit)	`_FftProcessingThread` (QThread, explicit)
Result deliver	`@Published` via `DispatchQueue.main.async`	Qt signals (`fftFrameReady`, `rmsLevelChanged`)
Audio deliver	`rawSampleHandler` `inputBuffer`	`queue.Queue` `raw_sample_handler`
Device selecti start	`audioEngine.start()`	`__init__` `set_device()`
Permis check	`AVCaptureDevice.authorizationStatus`	
File playba	`AVAudioPlayerNode`	`self.queue`
Device refresh	`loadAvailableInputDevices()`	`reinitialize_portaudio()`

Resource teardo	ARC / `deinit`	icit `close()`
iOS su		

_FftPro

Swift de

Python'

`_FftProcessingThread` `_input_buff`

(`_recent_peak_history`

accumu `TapTo` `lyzer`

dft_ana

Both are `vDSP_DFT_zrop`

(Accele

Python `scipy.fft.fft` `f` `fer`

via `window / sum(window)`

window

Signal l

- `rms_amp` `rmsLevelChanged`
  `inputLevelDB`
- `recent_peak_level_db`
  `recentPeakLevelDB` `tapPeakLevel`
- `fft_peak_amp` `peakMagnitude`
  Usec

Calibrat

Swift ap `performFFT(on:)` `computeFFT` `computeFFT`

pure. Py `_FftProcessingThread.run()` `dft_anal`

same pa `mag_y_db = mag_y_db + calibration`

Python `_FftProcessingThread.run()` `mag_y_db = mag_y_db + calibration`

Platforn

macOS `AudioObjectAddPropertyListener` `kAudioHardwarePropertyDevices`

Swift's `registerMacOSHardwareListener` `CM_Register_Notification`

cfgmgr3 `pyudev` `sound`

change `kAudioDevicePropertyNominalSampleRate`

requires polling every 5 s on Windows and Linux since neither platform provides a native per-device sample-rate-changed event.

Generated by Claude from GuitarTap Python source analysis · realtime_fft_analyzer.py + 3 modules